Assignment 2 - Perspective, Depth of Field, and Anaglyphs

In this assignment you will use projection frustums to draw a scene with blur due to depth of field, and to draw anaglyphs that allow for 3D viewing with inexpensive colour filter glasses.

Getting Started

There are new requirements to install for the assignment:

```
conda activate comp557f26
python -m pip install -r requirements.txt
```

See the `hello_triangle` and related examples shown in class for the basics of moderngl, qt5, and glm. The `pyglm` module documentation may be the most useful to consult:

- pyglm, note the "reference of PyGLM's functions", where you can search for functions, e.g., frustum
- moderngl, note the short introduction to OpenGL

The requirements also include `usd-core` which is used to load the sample scene and animation. USD stands for universal scene description, and is a powerful industry standard. The provided scene drawing code uses the format in a very minimal way (for the curious, see the USD docs).

Provided Code

The sample code has several viewing modes which you change by entering numbers 1 through 7, or by using the control panel.

1. The third person world view (default) with frustum visualizations.
2. The primary camera view.
3. The camera view with blurred depth of field.
4. The left eye view.
5. The right eye view.
6. The anaglyph view showing both left and right eyes in the same image.
7. The anaglyph view with depth of field blur.

When in the third person camera view, mouse drag gives turntable control of the view orientation (`Mode1World.world_view_orientation`), and the mouse scroll wheel moves the camera closer or further away from the scene (`Mode1World.world_viewing_distance`). When in all the other viewing modes, mouse drag and mouse scroll wheel change the primary camera's position (`RenderContext.scene_distance`) and scene orientation (`USDScene.scene_orientation`). Furthermore, holding shift or control while scrolling the mouse wheel will change the focal plane distance (`RenderContext.focal_distance`). Returning to the third person camera view, you will see the orientation and distance changes you made in the other view. Likewise, you will see that scene distance and focal plane distance are parameters you can edit directly in the control panel.

The primary goal of the assignment is to implement the viewing and projection transformations for the different viewing modes, and to implement a visualization in the third person camera view. The `RenderContext` contains all the parameters necessary to implement the viewing and projection transformations, and you will likely want to implement helper functions in that class to use in mode 1 for visualization, and in other modes for the actual viewing and projection transformations.

There is some extra code to help you with drawing frustums, rectangles, the depth of field blur effect, and the anaglyph effect. Here we also provide brief details about the scene drawing code.

- **`lines_helper.py`** provides a simple class to draw lines in 3D. You can use this class to draw the screen rectangle, viewing frustums, and coordinate frames. Recall you can use the inverse of a projection matrix as a modeling transformation.
- **`accum_buffer.py`** provides a render-to-texture implementation of an accumulation buffer. Mode 3 shows how to use this class to average multiple renders of the scene with different frustums to create a depth of field effect. You will need to implement the frustum offsets for this effect.
- **`fibonacci_disc_point`** in `render_context.py` is a function that provides a set of points on a disc of given radius that are evenly distributed. You will use these points to create different frustums for the depth of field effect.
- **`fbo.color_mask`** is used in viewing modes 4, 5, 6, and 7 to draw the left and right eye images into different colour channels of the frame buffer. When masking is enabled, only the requested colour channels are modified, permitting an anaglyph to be drawn with two render passes with different viewing and projection transforms. You will need to implement the correct frustums for the left and right eyes.
- **`usd_scene.py`** provides a class to load and draw a USD scene. The code assigns random pastel colours to the different objects in the scene, and uses a simple shader to draw the objects. The `paintGL` function takes projection and viewing matrices as input, and combines them with the local-to-world transformation for each object. This transformation includes animated positions and orientations, a global scene_matrix which scales and rotates the scene for viewing, and the `scene_orientation` matrix which is set by mouse drag in the primary camera views (2 through 7).

Note that because the paint method sets animation transforms with the current time, you will see a natural motion blur effect when using the depth of field viewing modes since the drawing is averaged over multiple frames.

Coordinate and Projection Conventions

All distances are measured in metres. The primary camera is at `(0, 0, scene_distance)` and looks along the negative z-axis. The screen rectangle lies in th xy plane, with the z-axis going through the center of the rectangle. It is located `screen_distance` metres in front of the camera. Thus, the screen is at z = -screen_distance in camera coordinates, and the world origin is at z = -scene_distance in camera coordinates. The focal plane is similarly located at z = -focal_distance in camera coordinates.

Convert the viewport width and height from pixels to metres using `METERS_PER_PIXEL`, defined in `render_context.py` (you may need to adjust this computation for ideal viewing of anaglyphs on your own computer). These values define the physical width and height of the screen rectangle. Compute the frustum bounds at the screen or focal plane, then scale them to the near plane before calling `glm.frustum(left, right, bottom, top, near_distance, far_distance)`. The viewing matrix must translate the scene by `-scene_distance`. For stereo rendering, place the left and right cameras at x-offsets `-interocular_distance / 2` and `+interocular_distance / 2`, respectively, and adjust each frustum so that both views converge on the same screen plane (though note this will not be the case for sample frustums for depth of field blur). Refer to the lecture slides for additional details on how to compute the frustum bounds for stereo rendering.

Steps and Objectives

In each step, you should be able to visually verify that your implementation is working correctly.

1. **Draw Screen Rectangle and Primary Camera Frame (1 mark)**
   Draw a 0.1 meter scaled coordinate frame with `LineHelper.draw_axis` at the position of the primary camera. Draw a `GREEN` rectangle in the xy plane centered at the location of the screen. The screen rectangle must be visualized as the current window's width and height in metres (see that the width and height in pixels are already read from `render_ctx.ctx.viewport` to set up the third person projection matrix).
2. **Draw Frustum and Viewing Mode 2 (2 marks)**
   Create the primary camera's frustum using `glm`. Use the screen dimensions defined above in the Coordinate and Projection Conventions section, and compute appropriate left, right, top, and bottom values at the near plane for the frustum. Use the `near_distance` and `far_distance` provided in the render context. Visualize the frustum in the third person camera view using the

`LineHelper` class, and use the frustum and the appropriate viewing transformation in the mode 2 `paint` method.

3. **Focal Plane Rectangle (1 mark)**
   Draw a `BLUE` rectangle to show the focal plane in mode 1, and use the `focal_distance` parameter to set the distance of the focal plane from the primary camera. Use `LineHelper.draw_wire_rect` with an appropriate scale and translation and modeling transformation to draw the rectangle (the rectangle should exactly fit inside the frustum).
4. **Shifted Frustums for Depth of Field (2 marks)**
   Use the `fibonacci_disc_point` function to obtain a set of points on a disc of radius `aperture_radius`. Use these points to create different camera positions and frustums for the primary camera, and visualize them in mode 1. You should see that the shifted frustums converge on the same focal-plane rectangle. Use the checkbox in the panel that sets `Mode1World.draw_all_sample_frustums` to decide if you should view all frustums, or only the zero offset frustum of the primary camera used in mode 2.
5. **Depth of Field Blur (1 mark)**
   Use the shifted frustums to render the scene multiple times, and average the results in an accumulation buffer. You should be able to see that objects at the focal plane distance are sharp, while objects closer or further away are blurred. Note that you may also notice a motion blur effect due to the animation of the scene.
6. **Draw Left and Right Eyes (1 mark)**
   This step can be done independently of objectives 3, 4, and 5. See the provided code helps you with the colour masking. Mode 1, with the `frustum_view_mode` set to `stereo_both` should show the left and right frustums drawn in red and cyan.
   Test that your frustums are correct by adjusting the near plane distance, screen distance, and interocular distance, and viewing the results in viewing modes 1, 4, and 5.
7. **Anaglyphs (1 mark)**
   For mode 6, combine the images you render for mode 4 and 5.
   You should be able to make objects appear both behind and in front of the screen (try adjusting the `scene_distance`). Be sure you do not have your left and right eyes of your viewing glasses reversed! The `interocular_distance` parameter value can be adjusted if you are finding it hard to fuse the anaglyphs (using smaller values might help).
8. **Anaglyph with Depth of Field (1 mark)**
   Combine your efforts in the previous step to create anaglyphs with depth of field blur.

Finished?

Great! Submit to MyCourses your python implementation and readme. Use a `zip` archive (only use `zip` format). Please do not submit any unnecessary files (e.g., *_pycache_* folder, .vscode folder, this

specification, usdc file, etc.).
Use the comments at the top of the files you submit to list your name and student number.

DOUBLE CHECK your submitted files by downloading them. You will not receive marks for assignments if files are missing, if it is a different assignment, or if the files are corrupt!

Recall the readme can be used to request a late penalty waiver following the mechanism described in the first lecture slides.

Note that you are encouraged to discuss assignments with your classmates, but not to the point of sharing code and answers. All code and written answers must be your own. Please see the course outline and the fine print in the slides of the first lecture.